

Chapter 2

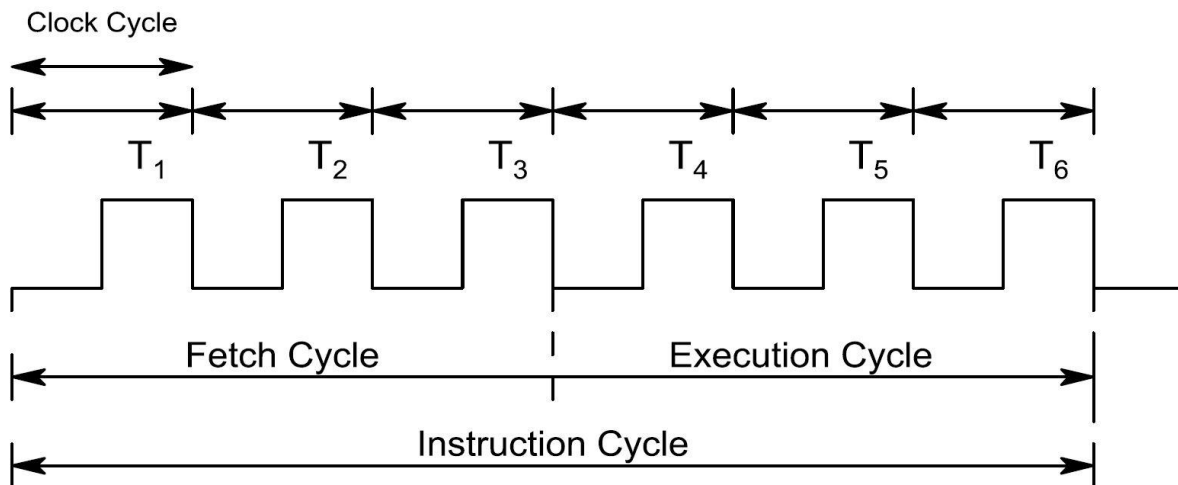
Assembly Language Programming

Instruction description and Format

The computer can be used to perform a specific task, only by specifying the necessary steps to complete the task. The collection of such ordered steps forms a 'program' of a computer. These ordered steps are the instructions. Computer instructions are stored in central memory locations and are executed sequentially one at a time. The control reads an instruction from a specific address in memory and executes it. It then continues by reading the next instruction in sequence and executes it until the completion of the program.

Instruction cycle

Instruction contains in the program and is pointed by the program counter. It is first moved to the instruction register and is decoded in binary form and stored as an instruction in the memory. The computer takes a certain period to complete this task i.e., instruction fetching, decoding and executing on the basis of clock speed. Such a time period is called 'Instruction cycle' and consists two cycles namely fetch and decode and Execute cycle. In the fetch cycle the central processing unit obtains the instruction code the memory for its execution. Once the instruction code is fetched from memory, it is then executed. The execution cycle consists the calculating the address of the operands, fetching them,



performing operations on them and finally outputting the result to a specified location.

An instruction manipulates the data and a sequence of instructions constitutes a program. Generally each instruction has two parts: one is the task to be performed, called the operation code (Op-Code) field, and the second is the data to be operated on, called the operand or address field. The operand (or data) can be specified in various ways. It may include 8-bit (or 16-bit) data, an internal register, a memory location, or an 8-bit (or 16-bit) address. The Op-Code field specifies how data is to be manipulated and address field indicates the address of a data item. For example:

ADD R₁, R₂

Op-code address

Here R_0 is the source register and R_1 is the destination register. The instruction adds the contents of R_0 with the content of R_1 and stores result in R_1 . 8085 A can handle at the maximum of 256 instructions (2^8) (246 instructions are used). The sheet which contains all these instructions with their hex code, mnemonics, descriptions and function is called an instruction sheet. Depending on the number of address specified in instruction sheet, the instruction format can be classified into the categories.

One address format (1 byte instruction)

Here 1 byte will be Op-code and operand will be default. E.g. *ADD B, MOV A, B*

Two address format (2 byte instruction)

Here first byte will be Op-code and second byte will be the operand/data. E.g. *IN 40H, MVI A, 8 – bit Data*

Three address format (3 byte instruction)

Here first byte will be Op-code, second and third byte will be operands/data. That is 2nd byte- lower order data. 3rd byte – higher order data

E.g. *LXI B, 4050 H*

Micro operation specifies the transfer of data into or out of a register.

Classification of an instruction

An instruction is a binary pattern designed inside a microprocessor to perform a specific function (task). The entire group of instructions called the instruction set. The 8085 instruction set can be classified into 5- different groups.

Data transfer group

The instructions which are used to transfer data from one register to another register or register to memory.

Arithmetic group

The instructions which perform arithmetic operations such as addition, subtraction, increment, decrement etc.

Logical group

The instructions which perform logical operations such as *AND*, *OR*, *XOR*, *COMPARE* etc.

Branching group

The instructions which are used for looping and branching are called branching instructions like *JUMP*, *CALL* etc.

Miscellaneous group

The instructions relating to stack operation, controlling purposes such as interrupt operations are fall under miscellaneous group including machine control like *HLT*, *NOP*.

1. Data Transfer Group Instructions

It is the longest group of instructions in 8085. This group of instruction copy data from a source location to destination location without modifying the contents of the source. The transfer of data may be between the registers or between register and memory or between an I/O device and accumulator. None of these instructions changes the flag. The instructions of this group are:

- I. *MOV R_d, R_s* (move register instruction)
 - 1 byte instruction
 - Copies data from source register to destination register. $R_d \leftarrow R_s$
 - R_d & R_s may be A, B, C, D, E, H & L
 - E.g. *MOV A, B*

II. *MVI R, 8 bit data* (move immediate instruction)

- 2 byte instruction
- Loads the second byte (8 bit immediate data) into the register specified.
- R may be A, B, C, D, E, H & L
- E.g. *MVI C, 53H* $C \leftarrow 53H$

III. *MOV M, R* (Move to memory from register)

- Copy the contents of the specified register to memory. Here memory is the location specified by contents of the HL register pair.
- E.g. *MOV M, B*

IV. *MOV R, M* (move to register from memory)

- Copy the contents of memory location specified by HL pair to specified register.
- E. g. *MOV B, M*

Write a program to load memory locations 7090 H and 7080 H with data 40H and 50H and then swap these data.

Soln :

MVI H, 70H

MVI L, 90H

MVI A, 40H

MOV M, A

MOV C, M

MVI L, 80H

MVI B, 50H

MOV M, B

MOV D, M

MOV M, C

MVI L, 90H

MOV M, D

HLT

V. *LXI R_p , 2 bytes data* (load register pair)

- 3-byte instruction
- Load immediate data to register pair
- Register pair may be BC, DE, HL & SP(Stack pointer)
- 1st byte- Op-code
- 2nd byte – lower order data
- 3rd byte- higher order data
- E.g. *LXI B, 4532H*; $B \leftarrow 45, C \leftarrow 32H$

VI. *MVI M , data* (load memory immediate)

- 2 byte instruction.
- Loads the 8-bit data to the memory location whose address is specified by the contents of HL pair.
- E.g. *MVI M, 35H*; $[HL] \leftarrow 35H$

VII. *LDA 4035H* (Load accumulator direct)

- 3-byte instruction
- Loads the accumulator with the contents of memory location whose address is specified by 16 bit address.
- $A \leftarrow [4035H]$

VIII. *LDAX R_p* (Load accumulator indirect)

- 1 byte instruction.
- Loads the contents of memory location pointed by the contents of register pair to accumulator.
- E. g. *LDAX B* $[A] \leftarrow [[BC]]$
- *LXI B, 9000H* $B \leftarrow 90, C \leftarrow 00$
- *LDAX B* $A \leftarrow [9000]$

IX. *STA 16 – bit address* (store accumulator contents direct)

- 3-byte instruction.
- Stores the contents of accumulator to specified address
- E.g. *STA FA00H* $[FA00] \leftarrow [A]$

X. *STAX R_p* $[R_p] \leftarrow A$

- Stores the contents of accumulator to memory location specified by the contents of register pair. 1 byte instruction
- E. g. *STAX B*
- *LXI B, 9500H* Output
- *LXI D, 9501H* $[9500] = 32$
- *MVI A, 32H* $[9501] = 7A$

- *STAX B*
- *MVI A, 7AH*
- *STAX D* $[DE] \leftarrow A$

XI. *IN 8 – bit address*

- 2-byte instruction
- Read data from the input port address specified in the second byte and loads data into the accumulator i.e. input port content to accumulator.
- E. g. *IN 40H* $A \leftarrow [40H]$

XII. *OUT 8 – bit address*

- 2-byte instruction
- Copies the contents of the accumulator to the output port address specified in the 2nd byte. That means accumulator to output port: P A
- E. g. *OUT 40H* $[40] \leftarrow A$

XIII. *LHLD 16 – bit address* (Load HL directly)

- 3-byte instruction.
- Loads the contents of specified memory location to L –register and contents of next higher location to H-register.
- E.g. *LXI H, 9500H*
- *MVI M, 32H*
- *MVI L, 01H*
- *MVI M, 7AH*
- *LHLD 9500H* $H=7A, L=32$

XIV. *SHLD 16 – bit address* (store HL directly)

- Opposite to LHLD.
- Stores the contents of L-register to specified memory location and contents of H-register to next higher memory location.
- E .g. *LXI H, 9500H* $[8500]=00$
- *SHLD 8500H* $[8501]=95$

XV. *XCHG* (Exchange)

- Exchanges DE pair with HL pair.
- E. g. *LXI H, 7500H* $H= 75, L=00$
- *LXI D, 9532H* $D=95. E=32$
- *XCHG* $H=95, L=32$
- $D=75 E=00$

2. Arithmetic Group Instructions

The 8085 microprocessor performs various arithmetic operations such as addition, subtraction, increment and decrement. These arithmetic operations have the following mnemonics.

- I. *ADD R/M*
 - 1-byte add instruction.
 - Adds the contents of register/memory to the contents of the accumulator and stores the result in accumulator.
 - E. g. *ADD B* $A \leftarrow [A] + [B]$
- II. *ADI 8 bit data*
 - 2-byte add immediate instruction.
 - Adds the 8 bit data with the contents of accumulator and stores result in accumulator.
 - E g. *ADI 9BH* $A \leftarrow A + 9BH$
- III. *SUB R/M*
 - 1-byte subtract instruction.
 - Subtracts the contents of specified register / m with the contents of accumulator and stores the result in accumulator.
 - E. g. *SUB D* $A \leftarrow A - D$
- IV. *SUI 8 bit data*
 - 2-byte subtract immediate instruction.
 - Subtracts the 8 bit data from the contents of accumulator stores result in accumulator.
 - E. g. *SUI D3H* $A \leftarrow A - D3H$
- V. *INR R/M, DCR R/M*
 - 1-byte increment and decrement instructions.
 - Increase and decrease the contents of R(register) or M(memory) by 1 respectively.
 - E. g. *DCR B* $B = B - 1$
 - *DCR M* $[HL] = [HL] - 1$
 - *INR A* $A = A + 1$
 - *INR M* $[HL] + 1$
 - For these, all flags are affected except carry.
- VI. *INX R_p, DCX R_p*
 - Increase and decrease the register pair by 1.
 - Acts as 16 bit counter made from the contents of 2 registers (1 byte instruction)
 - E.g. *INX B* $BC = BC + 1$
 - *DCX D* $DE = DE - 1$
 - No flags affected
- VII. *ADC R/M and ACI 8 – bit data (addition with carry (1 byte))*
 - ACI 8-bit data= immediate (2 byte).
 - Adds the contents of register or 8 bit data whatever used suitably with the previous carry.
 - E.g. *ADC B* $A \leftarrow A + B + CY$
 - *ACI 70H* $A \leftarrow A + 70 + CY$

VIII. *SBB B/M*

- 1-byte instruction.
- Subtracts the contents of register or memory from the contents of accumulator and stores the result in accumulator.
- g. *SBB D* $A \leftarrow A - D - \text{Borrow}$

IX. *SBI 8 bit data*

- 2-byte instruction.
- Subtracts the 8-bit immediate data from the content of the accumulator and stores the result in accumulator.
- E.g. *SBI 70H* $A \leftarrow A - 70 - \text{Borrow}$

X. *DAD R_p*(double addition)

- 1-byte instruction.
- Adds register pair with HL pair and store the 16 bit result in HL pair.
- E. g. *LXI H, 7320H*
- *LXI B, 4220H*
- *DAD B* $HL = HL + BC$
- $7320 + 4220 = B540$

XI. *DAA* (Decimal adjustment accumulator)

- Used only after addition.
- 1-byte instruction.
- The content of accumulator is changed from binary to two 4-bit BCD digits.
- E. g. *MVI A, 78H* $A = 78$
- *MVI B, 42H* $B = 42$
- *ADD B* $A = A + B = BA$
- *DAA* $A = 20, CY = 1$

The arithmetic operation add and subtract are performed in relation to the contents of accumulator.
The features of these instructions are

1. They assume implicitly that the accumulator is one of the operands.
2. They modify all the flags according to the data conditions of the result.
3. They place the result in the accumulator.
4. They do not affect the contents of operand register or memory.

But the INR and DCR operations can be performed in any register or memory. These instructions

- Affect the contents of specified register or memory.
- Affect the flag except carry flag.

3. Logical Group Instructions

A microprocessor is basically a programmable logic chip. It can perform all the logic functions of the hardwired logic through its instruction set. The 8085 instruction set includes such logic functions as AND, OR, XOR and NOT (Complement): The following features hold true for all logic instructions:

- The instructions implicitly assume that the accumulator is one of the operands.
- All instructions reset (clear) carry flag except for complement where flag remain unchanged.
- They modify Z, P & S flags according to the data conditions of the result.
- Place the result in the accumulator.
- They do not affect the contents of the operand register.

The logical operations have the following instructions.

I. *ANA R/M* (the contents of register/memory)

- Logically AND the contents of register/memory with the contents of accumulator.
- 1-byte instruction.
- CY flag is reset and AC is set.

II. *ANI 8 bit data*

- Logically AND 8 bit immediate data with the contents of accumulator.
- 2-byte instruction.
- CY flag is reset and AC is set. Others as per result

III. *ORA R/M*

- Logically OR the contents of register/memory with the contents of accumulator.
- 1-byte instruction.
- CY and AC is reset and other as per result.

IV. *ORI 8 bit data*

- Logically OR 8 bit immediate data with the contents of the accumulator.
- 2-byte instruction.

- CY and AC is reset and other as per result.

V. *XRA R/M*

- Logically exclusive OR the contents of register memory with the contents of accumulator.
- 1-byte instruction.
- CY and AC is reset and other as per result.

VI. *XRI 8 bit data*

- Logically Exclusive OR 8 bit data immediate with the content of accumulator.
- 2-byte instruction.
- CY and AC is reset and other as per result.

VII. *CMA* (Complement accumulator)

- 1-byte instruction.
- Complements the contents of the accumulator.
- No flags are affected.

Instruction	CY	AC
<i>ANA/ANI</i>	0	1
<i>ORA/ORI</i>	0	0
<i>XRA/XRI</i>	0	0

VIII. *CMP R/M* (1 byte instruction)

IX. *CPI 8 bit data* (2 byte instruction)

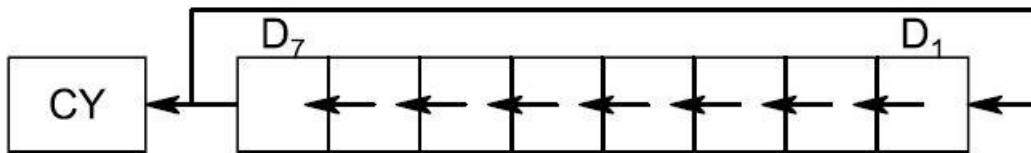
- Compare the contents of register/ memory and 8 bit data with the contents of accumulator.
- Status is shown by flags & all flags are modified.

Case	CY	Z	
$[A] < [R/M]$ or 8 bit	1	0	$A-R < 0$
$[A] = [R/M]$ or 8 bit	0	1	$A-R = 0$
$[A] > [R/M]$ or 8 bit	0	0	$A-R > 0$

Used to indicate end of data.

X. *RLC*: Rotate accumulator left

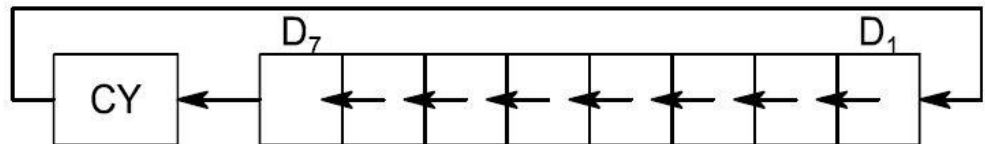
- Each bit is shifted to the adjacent left position. Bit D7 becomes D0.
- The carry flag is modified according to D7.



$$CY = D_7, D_7 = D_6, D_6 = D_5, \dots, D_1 = D_0, D_0 = D_7$$

XI. *RAL*: Rotate accumulator left through carry

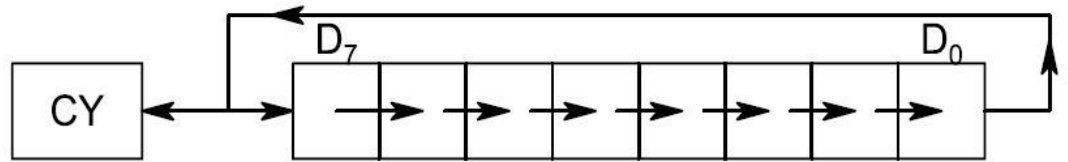
- Each bit is shifted to the adjacent left position. Bit D7 becomes the carry bit and the carry bit is shifted into D0.
- The carry flag is modified according to D7.



$$CY = D_7, D_7 = D_6, D_6 = D_5, \dots, D_1 = D_0, D_0 = CY$$

XII. *RRC*: rotate accumulator right

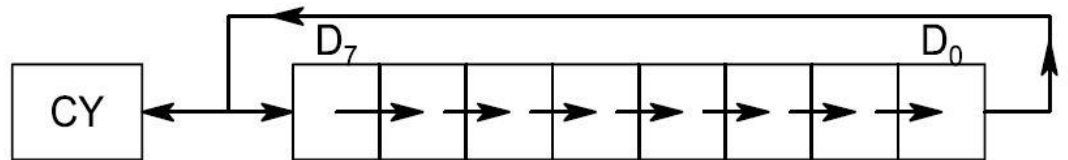
- Each bit is shifted right to the adjacent position. Bit D0 becomes D7.
- The carry flag is modified according to D0.
- The carry flag is modified according to D0.



$$CY = D_0, D_7 = D_0, \dots, D_0 = D_1$$

XIII. *RAR*: Rotate accumulator right through carry

- Each bit is shifted right to the adjacent position. Bit D0 becomes the carry bit and the carry bit is shifted into D7.



$$CY = D_0, D_7 = D_0, \dots, D_0 = D_1$$

4. Branching Group Instructions

The microprocessor is a sequential machine; it executes machine codes from one memory location to the next. The branching instructions instruct the microprocessor to go to a different memory location and the microprocessor continues executing machine codes from that new location. The branching instructions are the most powerful instructions because they allow the microprocessor to change the sequence of a program, either unconditionally or under certain test conditions. The branching instruction code categorized in following three groups:

- Jump instructions
- Call and return instruction
- Restart instruction

Jump Instructions

The jump instructions specify the memory location explicitly. They are 3 byte instructions, one byte for the operation code followed by a 16 bit (2 byte) memory address. Jump instructions can be categorized into unconditional and conditional jump.

Unconditional Jump

8085 includes unconditional jump instruction to enable the programmer to set up continuous loops without depending only type of conditions. E.g. JMP 16 bit address: loads the program counter by 16 bit address and jumps to specified memory location.

E.g. JMP 4000H.

Here, 40H is higher order address and 00H is lower order address. The lower order byte enters first and then higher order.

The jump location can also be specified using a label or name.

E.g.

MVI A, 80H	START: IN 00H
OUT 43H	OUT 01H
MVI A, 00H	JMP START
L1: OUT 40H	
INR A	
JMP L1	
HLT	

Mnemonics	Description
JC 16 bit	Jump on carry (if CY=1)
JNC 16 bit	Jump on if no carry (if CY=0)
JZ 16bit	Jump on zero (if Z=1)
JNZ 16bit	Jump on if no zero (if Z=0)
JP 16bit	Jump on positive (if S=0)
JM 16bit	Jump on negative (if S=1)
JPE 16bit	Jump on parity even (if P=1)
JPO 16bit	Jump on parity odd (if P=0)

Call and return instructions: (Subroutine)

Call and return instructions are associated with subroutine technique. A subroutine is a group of instructions that perform a subtask. A subroutine is written as a separate unit apart

from the main program and the microprocessor transfers the program execution sequence from main program to subroutine whenever it is called to perform a task. After the completion of subroutine task microprocessor returns to main program. The subroutine technique eliminates the need to write a subtask repeatedly, thus it uses memory efficiently. Before implementing the subroutine, the stack must be defined; the stack is used to store the memory address of the instruction in the main program that follows the subroutines call.

To implement subroutine there are two instructions CALL and RET.

I. *CALL 16 bit memory*

- Call subroutine unconditionally.
- 3-byte instruction.
- Saves the contents of program counter on the stack pointer. Loads the PC by jump address (16 bit memory) and executes the subroutine.

II. *RET*

- Returns from the subroutine unconditionally.
- 1-byte instruction
- Inserts the contents of stack pointer to program counter.

III. *CC, CNC, CZ, CNZ, CP, CM, CPE, CPO*

- Call subroutine conditionally.
- Same as CALL except that it executes on the basis of flag conditions.

IV. *RC, RNC, RZ, RNZ, RP, RM, RPE, RPO*

- Return subroutine conditionally.
- Same as RET except that it executes on the basis of flag conditions.

V. Restart Instruction

8085 instruction set includes 8 restart instructions (RST). These are 1 byte instructions and transfer the program execution to a specific location.

Restart instruction	Hex Code	Call location in hex
RST 0	C7	0000H
RST 1	CF	0008H
RST 2	D7	0010H
RST 3	DF	0018H
RST 4	E7	0020H
RST 5	EF	0028H
RST 6	F7	0030H
RST 7	FF	0038H

When RST instruction is executed, the 8085 stores the contents of PC on SP and transfers the program to the restart location. Actually these restart instructions are inserted through additional hardware. These instructions are part of interrupt process.

5. Miscellaneous Group Instructions

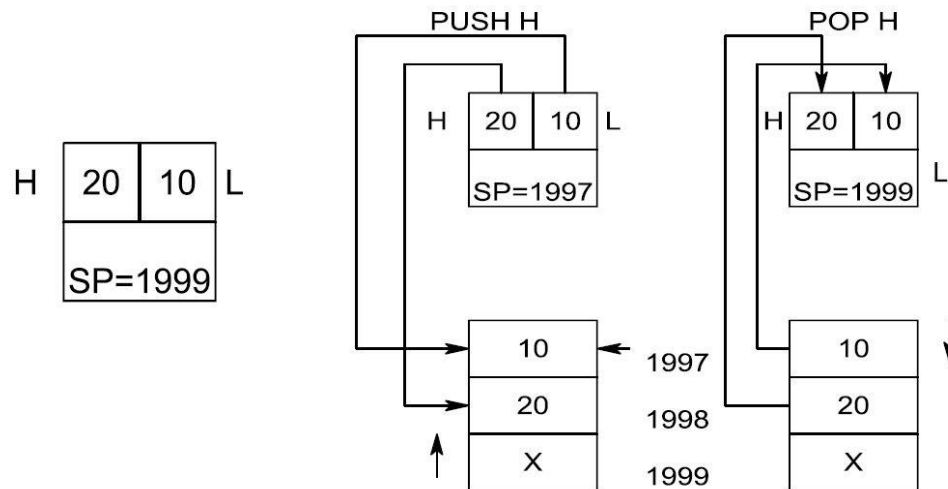
STACK

The stack is defined as a set of memory location in R/W memory, specified by a programmer in a main memory. These memory locations are used to store binary information temporarily during the execution of a program. The beginning of the stack is defined in the program by using the instruction LXI SP, 16 bit address. Once the stack location is defined, it loads 16 bit address in the stack pointer register. Storing of data bytes for this operation takes place at the memory location that is one less than the address e.g. LXI SP, 2099H

Here the storing of data bytes begins at 2098H and continuous in reverse order i.e 2097H. Therefore, the stack is initialized at the highest available memory location to prevent the program from being destroyed by the stack information. The stack instructions are:

- I. *PUSH Rp/PSW* (Store register pair on stack)
 - 1-byte instruction.
 - Copies the contents of specified register pair or program status word (accumulator and flag) on the stack.
 - Stack pointer is decremented and content of high order register is copied. Then it is again decremented and content of low order register is copied.
- II. *POP Rp/PSW* (retrieve register pair from stack)
 - 1-byte instruction.

- Copies the contents of the top two memory locations of the stack into specified register pair or program status word.
- A content of memory location indicated by SP is copied into low order register and SP is incremented by 1. Then the content of next memory location is copied into high order register and SP is incremented by 1.



- III. XTHL – exchanges top of stack (TOS) with HL
- IV. SPHL – move HL to SP
- V. PCHL – move HL to PC

Some instructions related to interrupt

DI – disable interrupt

EI – Enable interrupt

SIM – set interrupt mask

RIM – read interrupt mask

Addressing modes

Instructions are command to perform a certain task in microprocessor. The instruction consists of op-code and data called operand. The operand may be the source only, destination only or both of them. In these instructions, the source can be a register, a memory or an input port. Similarly, destination can be a register, a memory location, or an output port. The various format (way) of specifying the operands

are called addressing mode. So addressing mode specifies where the operands are located rather than their nature. The 8085 has 5 addressing mode:

1. Direct addressing mode

The instruction using this mode specifies the effective address as part of instruction. The instruction size either 2-bytes or 3-bytes with first byte op-code followed by 1 or 2 bytes of address of data.

E. g. *LDA 9500H* $A \leftarrow [9500]$
 IN 80H $A \leftarrow [80]$

This type of addressing is called absolute addressing.

2. Register Direct addressing mode

This mode specifies the register or register pair that contains the data.

E g. *MOV A, B*

Here register B contains data rather than address of the data. Other examples are: *ADD, XCHG* etc.

3. Register Indirect addressing mode

In this mode the address part of the instruction specifies the memory whose contents are the address of the operand. So in this type of addressing mode, it is the address of the address rather than address itself. (One operand is register)

e. g. *MOV R, M* *MOV M, R* *STAX, LDAX* etc.
 STAX B B= 95 C =00
 [9500] $\leftarrow A$

4. Immediate addressing mode

In this mode, the operand position is the immediate data. For 8-bit data, instruction size is 2 bytes and for 16 bit data, instruction size is 3 bytes.

E.g. *MVI A, 32H*
LXI B, 4567H

5. Implied or Inherent addressing mode

The instructions of this mode do not have operands.

E.g. *NOP*: No operation

HLT: Halt

EI: Enable interrupt

DI: Disable interrupt

Machine cycles and bus timing diagrams

Operation of a microprocessor can be classified in to following four groups according to their nature.

- Op- Code fetch
- Memory Read /Write
- I/O Read/ Write
- Request acknowledgement
- Here Op-Code fetch is an internal operation and other three are external operations.
- During three operations, microprocessor generates and receives different signals. These all operations are terms as machine cycle.
- Clock Cycle (T state): It is defined as one subdivision of the operation performed in one clock period.
- Machine Cycle: It is defined as the time required to complete one operation of accessing memory, I/O, or acknowledging an external request. This cycle may consist of three to six T-states.
- Instruction Cycle: It is defined as the time required completing the execution of an instruction. The 8085 instruction cycle consists of one to six machine cycles or one to six operations.

Op-Code fetch Machine Cycle

The first operation in any instruction is Op-Code fetch, The microprocessor needs to get(fetch) this machine code from the memory register where it is stored before the microprocessor can begin to execute the instruction.

Let's consider the instruction MOV C, A stored at memory location 2005H. The Op-Code for the instruction is 4FH and Op-Code fetch cycle is of 4 clock cycles.

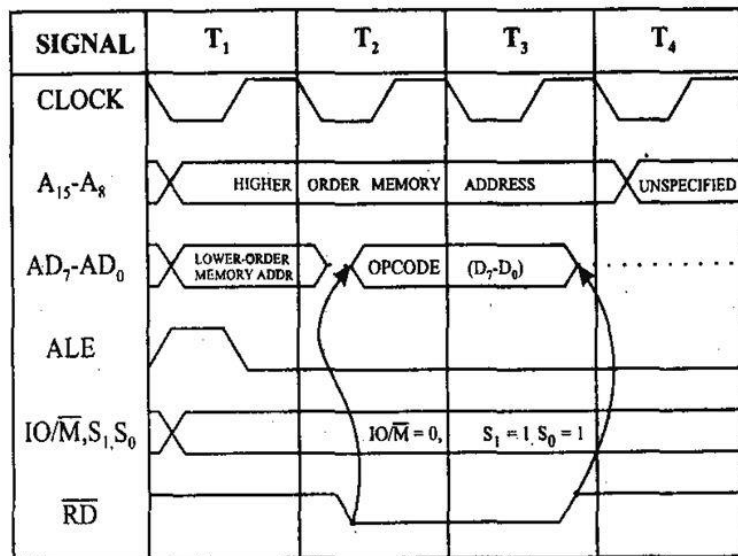


Fig: Timing diagram for Opcode fetch machine cycle

Step1: Microprocessor places the 16 bit memory address from Program Counter on the address bus. At T₁, high order address (20) is placed at A8-A15 and lower order address (05) is placed at

AD₀- AD₇ ALE signal goes high. IO/M goes low and both S₀ and S₁ goes high for Op-Code fetch.

Step 2: The control unit sends the control signal $\overline{RD}$ to enable the memory chip and active during T₂ and T₃.

Step 3: The byte from the memory location is placed on the data bus .that is 4f into D₀-D₇ and

$\overline{RD}$ goes high impedance.

Step4: The instruction 4FH is decoded and content of accumulator will be copied into register C during clock cycle T₄.

Memory Read Cycle:

Let's consider the instruction MVI A, 32 H stored at memory location 2000H.

2000 3EH MVI A, 32 H

2001 32H

Here two machine cycles are presented, first is Op-Code fetch which consists of 4 clock cycles and second is memory read consist of 4 clock cycle.

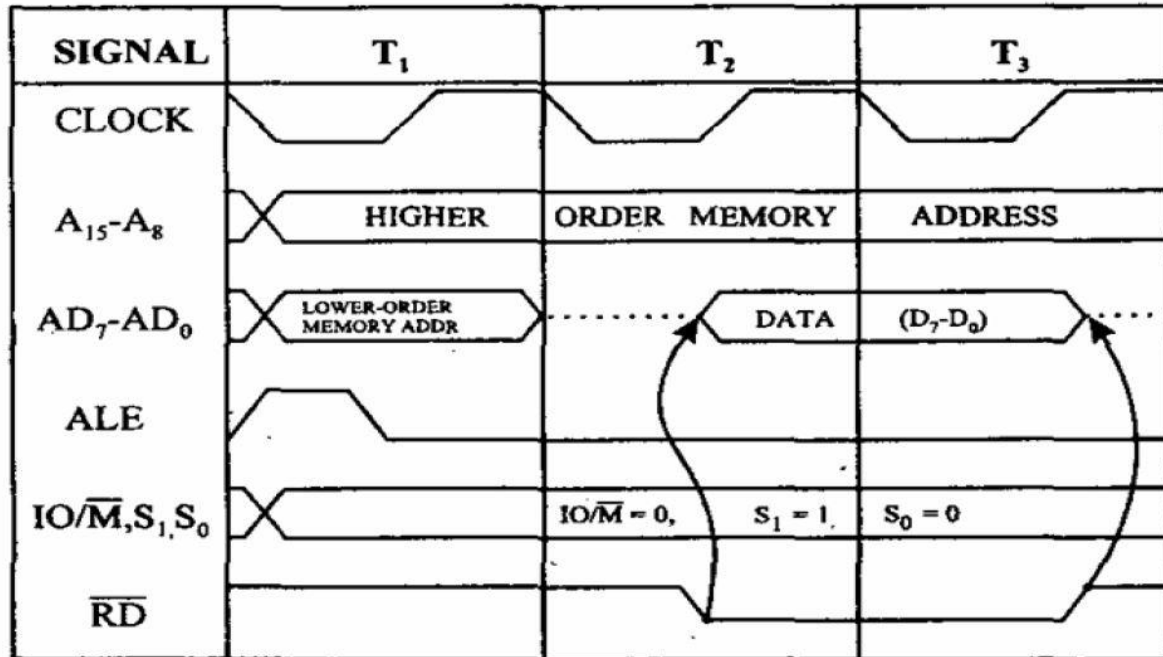


Fig - Timing Diagram for Memory Read Machine Cycle

Step 1 : First machine cycle (Op-Code fetch) is identical for timing diagram of Op-Code fetch cycle.

Step 2: After completion of Op-Code fetch cycle, 8085 places the address 2001 on the address bus and increments PC to 2002H. ALE is asserted high $IO/\overline{M} = 0, S_1 = 1, S_0 = 0$ for memory read cycle. When $\overline{RD} = 0$, memory places the data byte 32H on the data bus.

Memory Write Machine

- The memory write machine cycle is executed by the processor to write a data byte in a memory location.
- The processor takes, 3T states to execute this machine cycle.

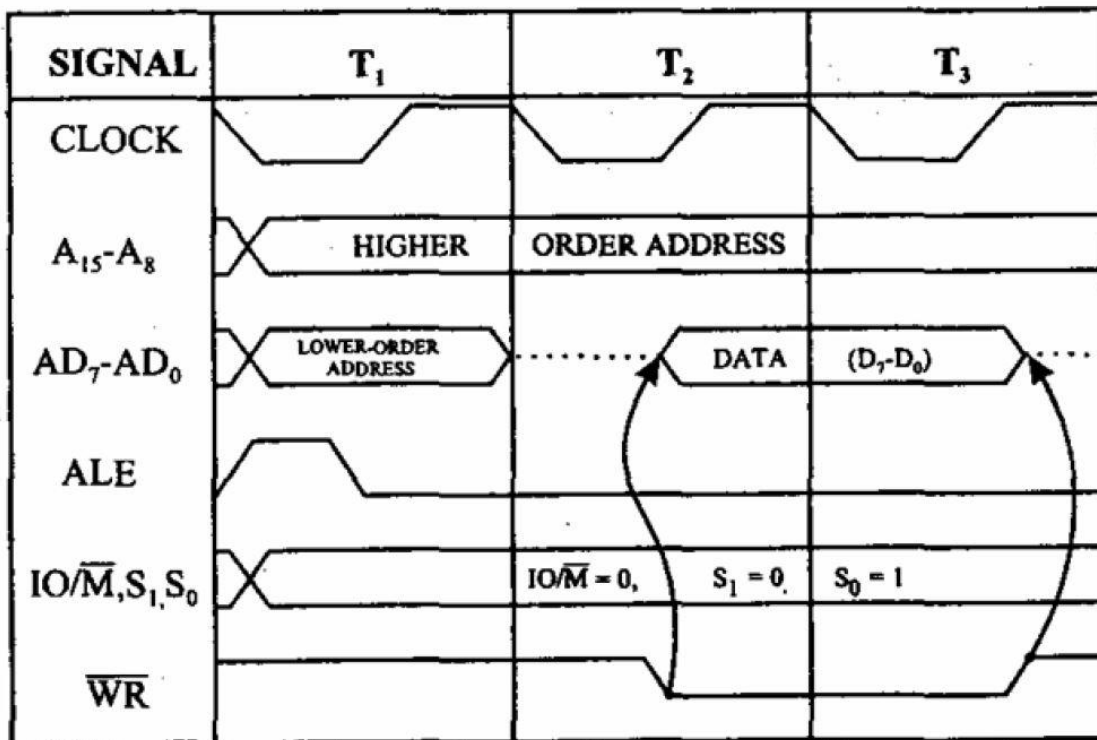


Fig - Timing Diagram for Memory Write Machine Cycle

I/O Read Cycle:

- The I/O Read cycle is executed by the processor to read a data byte from I/O port or from the peripheral, which is I/O, mapped in the system.
- The processor takes 3T states to execute this machine cycle.
- The IN instruction uses this machine cycle during the execution.

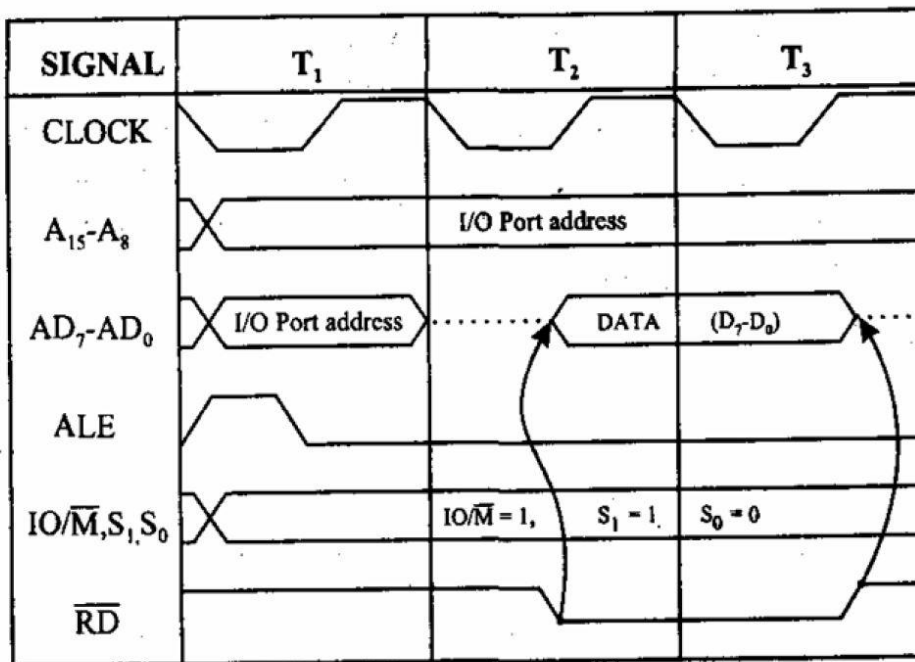


Fig - Timing Diagram for I/O Read Machine Cycle

I/O Write Cycle:

Let's consider the instruction OUT 01H stored at memory location 2050H.

2050	D3	OUT 01H
2051	01	

Op-Code Fetch Cycle	4T
Memory read Cycle	3T
I/O Write Cycle	3T

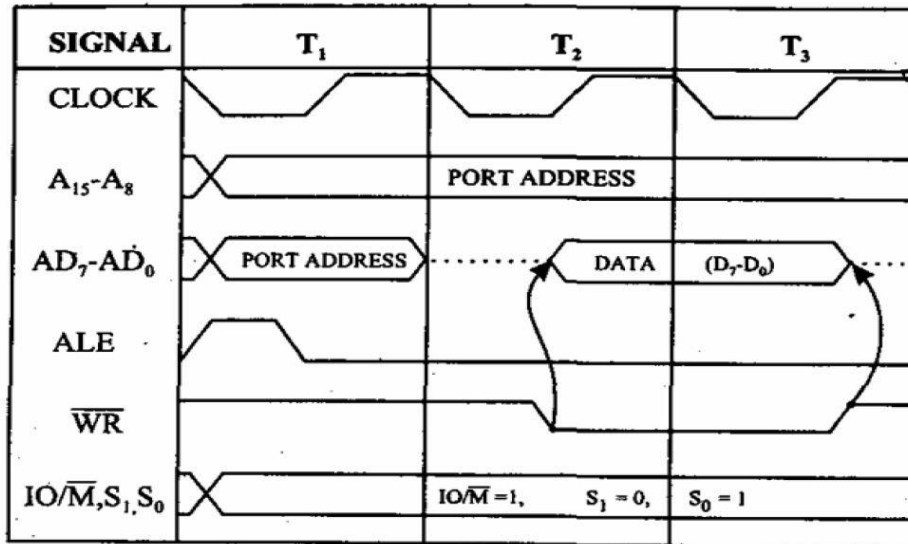


Fig - Timing Diagram for I/O Write Machine Cycle

Step 1: In Machine Cycle M_1 , the microprocessor sends $\overline{RD}$ control signal which is combined with $IO/\overline{M}$ to generate the $\overline{MEMR}$ signal and processor fetches instruction code D3 using the data bus.

Step 2: In 2nd Machine Cycle M_2 , the 8085 microprocessor places the next address 2051 on the address bus and gets the device address 01H via data bus.

Step 3: In machine Cycle M_3 , the 8085 places device address 01H on low order as well as high order address bus. $IO/\overline{M}$ goes high for IO and accumulator content are placed on Data bus which are to be written into the selected output port.

Timing diagram for STA 526AH

- STA means Store Accumulator -The contents of the accumulator is stored in the specified address (526A).
- The opcode of the STA instruction is said to be 32H. It is fetched from the memory 41FFH (see fig). - *of machine cycle*
- Then the lower order memory address is read (6A). - *Memory Read Machine Cycle*
- Read the higher order memory address (52).- *Memory Read Machine Cycle*
- The combination of both the addresses are considered and the content from accumulator is written in 526A. - *Memory Write Machine Cycle*
- Assume the memory address for the instruction and let the content of accumulator is C7H. So, C7H from accumulator is now stored in 526A.

Address	Mnemonics	Op code
41FF	STA 526AH	32 _H
4200		6A _H
4201		52 _H

